

TEMA 096. PROCESOS DE PRUEBAS Y GARANTÍA DE CALIDAD EN EL DESARROLLO DE SOFTWARE. PLANIFICACIÓN, ESTRATEGIA DE PRUEBAS Y ESTÁNDARES. NIVELES, TÉCNICAS Y HERRAMIENTAS DE PRUEBAS DE SOFTWARE. CRITERIOS DE ACEPTACIÓN DE SOFTWARE.

Actualizado a 10/01/2022

1 INTRODUCCIÓN

El proceso de pruebas aplicado en el desarrollo, mantenimiento y operación del software reduce riesgos, contribuye a la calidad del sistema y ayuda a verificar requisitos contractuales, legales y estándares específicos del sector.

Las pruebas aportan fiabilidad y calidad software.

Las actividades de pruebas se dan antes y después de la ejecución de la prueba:

- Planificar y controlar.
- Seleccionar las condiciones de las pruebas.
- Diseñar y ejecutar casos de prueba.
- Comprobar resultados.
- Evaluar criterios de salida.
- Elaborar informes sobre el proceso de pruebas y el sistema probado.
- Finalizar o completar actividades de cierre tras finalizar una fase de prueba.
- Revisión de documentos (incluye código fuente).
- Relación de análisis estáticos.

Existen distintos niveles, y tipos de pruebas, dinámicas y estáticas, generando información para mejorar tanto el sistema probado como el desarrollo y los procesos de prueba. Para llevar a cabo estas pruebas existen diferentes herramientas.

El coste de eliminar defectos se incrementa con el tiempo durante el cual el defecto permanece en el sistema. La identificación de errores en etapas tempranas permite la corrección de los mismos a costos reducidos.

Cuanto más temprana es la detección de un defecto, menos costosa es su corrección

Conceptos Bug, defecto, error, fallo (ISTQB)

Una persona puede cometer un **error**(equivocación) que puede producir un **defecto (falta, bug)** en el código de programa o en un documento. Si se ejecuta un defecto en el código, el sistema puede no hacer lo que se espera o hacer algo que no debería, lo que provocaría un **fallo**.

Error humano → Defecto (Bug) → Fallo

Error agentes externos → Fallo

1.1 PRINCIPIOS DE PRUEBAS (ISTQB)

- Principio 1 – Las pruebas demuestran la presencia de defectos
- Principio 2 – Las pruebas exhaustivas no existen
- Principio 3 – Pruebas tempranas
- Principio 4 – Agrupación de defectos
- Principio 5 – Paradoja del pesticida
- Principio 6 – Las pruebas dependen del contexto
- Principio 7 – Falacia de ausencia de errores

2 ASEGURAMIENTO DE LA CALIDAD Y PROCESOS DE PRUEBAS (ISTQB)

Aseguramiento de la calidad del software (SQA) es un proceso que persigue que todos los procesos, métodos, actividades y elementos de trabajo de ingeniería de software sean observados y cumplan con los estándares definidos.

El proceso básico de pruebas consta de las siguientes actividades principales:

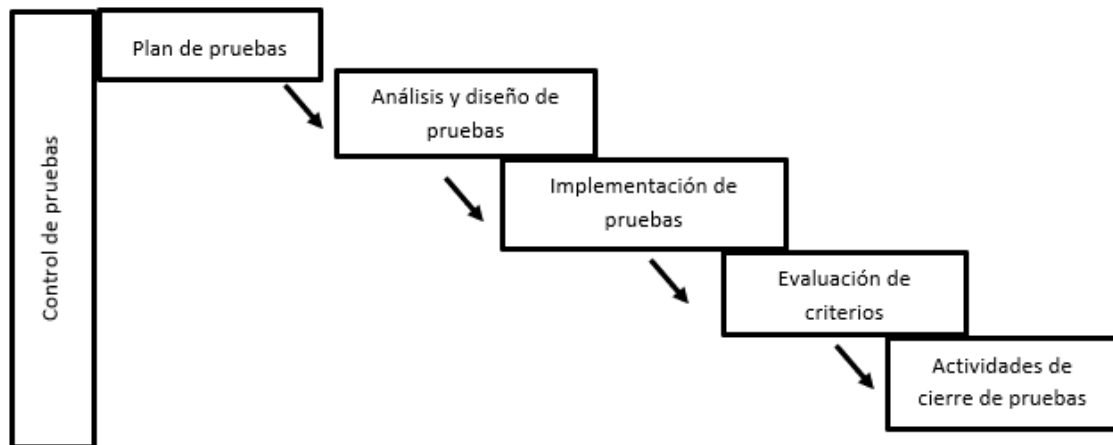


Imagen 1: Proceso de pruebas ISTQB

Las actividades del proceso pueden solaparse o realizarse a la vez. Es necesario adaptar estas actividades al contexto del sistema y del proyecto.

3 PLANIFICACIÓN (ISTQB)

El proceso de pruebas no es un proceso aislado, las actividades de pruebas están asociadas a actividades de desarrollo de software. Distintos modelos de ciclo de vida de desarrollo requieren distintos enfoques de las tareas de validación y verificación del aseguramiento de la calidad.

- **Verificación** Conjunto de actividades que aseguran que el software implementa correctamente una función específica. (Pressman). Da respuesta a la pregunta ¿estamos construyendo el producto correctamente? (Boehm).
¿El software está de acuerdo con su especificación?
- **Validación** Conjunto diferente de tareas que aseguran que el software que se construye sigue los requerimientos del cliente. (Pressman) Da respuesta a la pregunta ¿estamos construyendo un producto correcto? (Boehm).
¿El software cumple las expectativas del cliente?

Modelo en V

Es el modelo de desarrollo software más utilizado. Desarrollo y pruebas tienen dos ramas iguales. Cada nivel de desarrollo tiene su correspondiente nivel de pruebas. Las pruebas (rama derecha) se diseñan en paralelo al desarrollo software (rama izquierda). Las actividades del proceso de pruebas tienen lugar a lo largo de todo el ciclo de vida software. Cada nivel de desarrollo se verifica respecto de los contenidos del nivel que le precede.

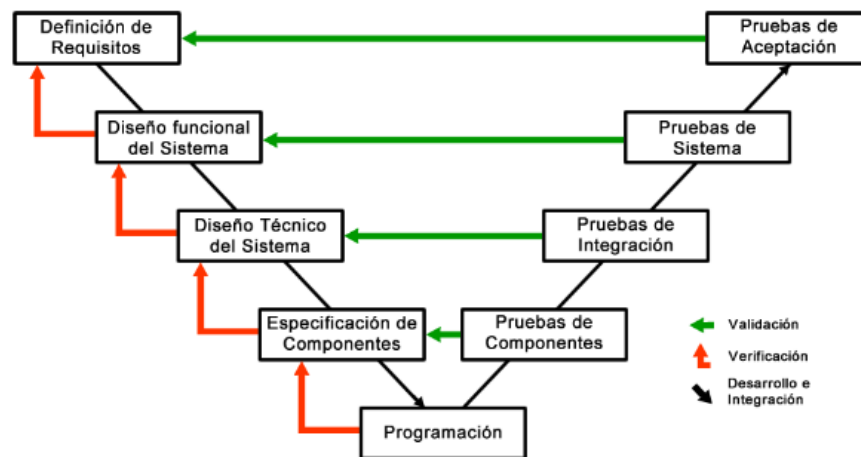


Imagen 2: Modelo en V

Modelo en W

El modelo W puede ser visto como una extensión del modelo en V, con la diferencia de que, en los estados del modelo en W, ciertas actividades de control de calidad se realizarán en paralelo con el proceso de desarrollo.

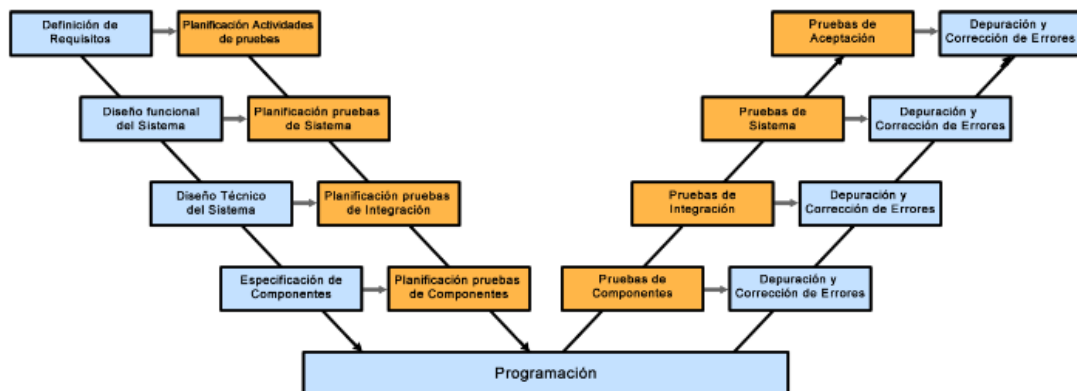


Imagen 3: Modelo en W

En los ciclos de vida Iterativo- incremental, Prototipo, RAD, RUP o Agile, la planificación de las tareas de verificación y validación deberá adaptarse ya que el producto es diseñado y construido mediante muchas entregas e iteraciones.

Las siguientes actividades forman parte de la planificación de pruebas:

- Estrategia de pruebas (test strategy).
- Planificación de recursos (resource planning).
- Prioridad de las pruebas (priority of tests).
- Soporte de herramientas (tool support).

4 ESTRATEGIA

Existen distintas prácticas de aplicación del proceso de pruebas software que afectan directamente al proceso de desarrollo.

4.1 TDD (TEST DRIVEN DEVELOPMENT)

TDD o desarrollo guiado por pruebas de software, o Test-driven development (TDD) es una práctica de ingeniería de software que involucra otras dos prácticas:

- Escribir las pruebas primero (Test First Development) basándose en los requisitos y se verifica que las pruebas fallen.
- Escribir el código fuente que pase la prueba satisfactoriamente.
- Refactorización del código escrito (Refactoring).



Imagen 4: Ciclo TDD

4.2 BDD (BEHAVIOR DRIVEN DEVELOPMENT)

BDD (Behavior Driven Development), es una estrategia de desarrollo dirigido por comportamiento, que ha evolucionado desde TDD (Test Driven Development), aunque no se trata de una técnica de pruebas como tal.

La principal diferencia es que todas las definiciones BDD se escriben en un idioma común. El principal objetivo es que el equipo describa los detalles de cómo se debe comportar la aplicación a desarrollar, y de esta forma será comprensible por todos.

Requiere utilizar la fórmula 'Given-When-Then' u otras como las historias de usuario 'Role-Feature-Reason'.

'Given-When-Then'

- Given 'dado': Se especifica el escenario, las precondiciones.
- When 'cuando': Las condiciones de las acciones que se van a ejecutar.
- Then 'entonces': El resultado esperado, las validaciones a realizar.

'Role-Feature-Reason'

- As a '**Como**': Se especifica el tipo de usuario.
- I want '**deseo**': Las necesidades que tiene.
- So that '**para que**': Las características para cumplir el objetivo.

La herramienta más destacada, basado en el patrón ‘Given-When-Then’ es Cucumber, un framework de test con soporte para BDD. En Cucumber, las especificaciones de BDD están escritas en lenguaje Gherkin, basado en ‘Given-When-Then’.

Hay muchas otras herramientas, tantas como lenguajes de programación:

- Java: JBehave, JDave, Instinct, beanSpec
- C: CSpec
- C#: NSpec, NBehave
- .NET: NSpec, NBehave, SpecFlow
- PHP: PHPSpec, Behat
- Ruby: RSpec, Cucumber
- JavaScript: JSSpec, jspec
- Python: Freshen

4.3 AUTOMATIZACIÓN DE PRUEBAS Y TESTING CONTINUO

Realizar pruebas continuas (Continuous Testing) es tanto una práctica como una filosofía dentro de los equipos de desarrollo de software. Los desarrolladores y los especialistas en garantizar la calidad de las aplicaciones inician la práctica del testing continuo cuando mediante un Pipeline CI/CD (Continuous Integration / Continuous Deployment), activan una lista de pruebas automatizadas que se ejecutan con cada construcción y entrega. Continuous Testing incluye además pruebas automáticas que permiten detectar problemas en producción.

Las herramientas de pruebas actuales se integran con herramientas CI /CD como Jenkins, CircleCI, Bamboo. En función del nivel de prueba a automatizar existen diferentes herramientas como SmartBear, BlazeMeter, Tricentis qTest, BrowserStack, SauceLabs, Postman, SOAPUI, Selenium, Cypress, Junit o analizadores de código como SonarQube.

5 ESTÁNDARES

Existen multitud de estándares relacionados con el proceso de pruebas y la garantía de calidad en el desarrollo de software. Se muestra una evolución de los más relevantes:

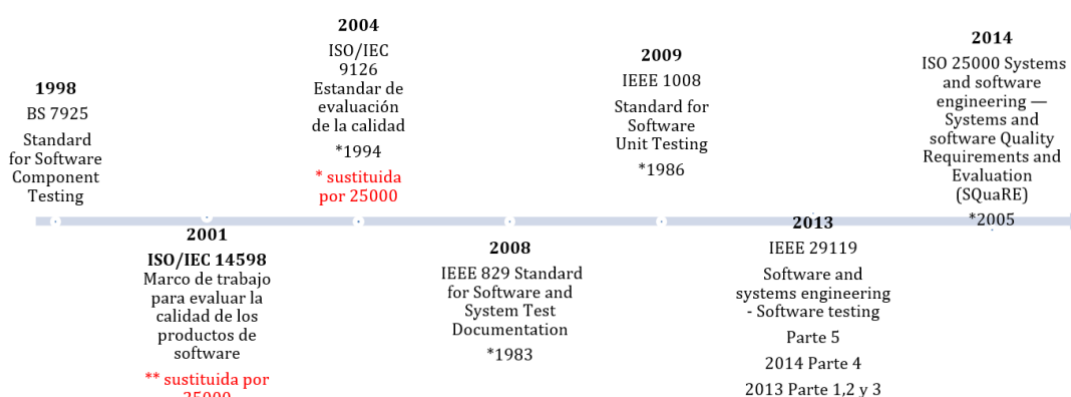


Imagen 5: Evolución Estándares de Pruebas Software y Calidad Software

Además, existen multitud de marcos de referencia o buenas prácticas que guardan relación con los procesos de pruebas y garantía de calidad en el desarrollo de software, como:

- Moprosoft: modelo de Referencia de Procesos conformado por un conjunto de buenas prácticas y procesos de gestión e ingeniería de software.
- CMMI, Capability Maturity Model Integration, conjunto de buenas prácticas organizadas por capacidades críticas de negocio con el objetivo de mejorar su rendimiento.
- SWEBOK, Software Engineering Body of Knowledge, es un documento creado por la Software Engineering Coordinating Committee, promovido por el IEEE.
- ITIL, Information Technology Infrastructure Library, biblioteca de infraestructura y tecnologías de la información, es un conjunto de conceptos y mejores prácticas referentes a la gestión de servicios TI.
- ISTQB, International Software Testing Qualifications Board, comité Internacional de Certificaciones de Pruebas de Software.
- Métrica 3, Metodología de Planificación, Desarrollo y Mantenimiento de Sistemas de Información creada por el Ministerio de Administraciones Públicas, que contempla prácticas de pruebas y una interfaz para el aseguramiento de la calidad.

5.1 ISO/IEC /IEEE 29119 SOFTWARE TESTING STANDARD

Su objetivo es cubrir todo el ciclo de vida de las pruebas de sistemas software incluyendo los aspectos relativos a la organización, gestión, diseño y ejecución de las pruebas, para reemplazar varios estándares IEEE y BSI sobre pruebas de software. Su estructura consta de cinco partes, un estándar relativo a revisiones y un modelo de evaluación basado en los procesos de la parte 2.

5.2 ISO 25000:2014

El conjunto de normas ISO 25000:2014 conocidas como SQuaRE (System and Software Quality Requirements and Evaluation) (Evaluación de Calidad de Productos Software).

El conjunto de normas ISO 25000:2014 provienen de la **evolución de las normas ISO/IEC 9126**, estándar de evaluación de la calidad de un software, e **ISO/IEC 14598**, marco de trabajo para evaluar la calidad del software.

Se divide en 6 secciones: ISO/IEC 2500n, ISO/IEC 2501n, ISO/IEC 2502n, ISO/IEC 25030, ISO/IEC 25040, e ISO/IEC 25050-25099.

El modelo de calidad del producto definido por la ISO/IEC 25010 se encuentra compuesto por las ocho características de calidad que se muestran en la siguiente figura:



Imagen 6: Características del producto Software - ISO/IEC 25010 Fuente: iso25000.com

5.2.1 ADECUACIÓN FUNCIONAL

Representa la capacidad del producto software para proporcionar funciones que satisfacen las necesidades declaradas e implícitas, cuando el producto se usa en las condiciones especificadas. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Completitud funcional.**
- **Corrección funcional.**
- **Pertinencia funcional.**

5.2.2 EFICIENCIA DE DESEMPEÑO

Esta característica representa el desempeño relativo a la cantidad de recursos utilizados bajo determinadas condiciones. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Comportamiento temporal.**
- **Utilización de recursos.**
- **Capacidad.** Compatibilidad.

Capacidad de dos o más sistemas o componentes para intercambiar información y/o llevar a cabo sus funciones requeridas cuando comparten el mismo entorno hardware o software. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Coexistencia.**
- **Interoperabilidad.**

5.2.3 USABILIDAD

Capacidad del producto software para ser entendido, aprendido, usado y resultar atractivo para el usuario, cuando se usa bajo determinadas condiciones. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Capacidad para reconocer su adecuación.**
- **Capacidad de aprendizaje.**
- **Capacidad para ser usado.**
- **Protección contra errores de usuario.**
- **Estética de la interfaz de usuario.**
- **Accesibilidad.**

5.2.4 FIABILIDAD

Capacidad de un sistema o componente para desempeñar las funciones especificadas, cuando se usa bajo unas condiciones y periodo de tiempo determinados. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Madurez.**
- **Disponibilidad.**
- **Tolerancia a fallos.**
- **Capacidad de recuperación.**

5.2.5 SEGURIDAD

Capacidad de protección de la información y los datos de manera que personas o sistemas no autorizados no puedan leerlos o modificarlos. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Confidencialidad.**

- **Integridad.**
- **No repudio.**
- **Responsabilidad.**
- **Autenticidad.**

5.2.6 MANTENIBILIDAD

Esta característica representa la capacidad del producto software para ser modificado efectiva y eficientemente, debido a necesidades evolutivas, correctivas o perfectivas. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Modularidad.**
- **Reusabilidad.**
- **Analizabilidad.**
- **Capacidad para ser modificado.**
- **Capacidad para ser probado.**

5.2.7 PORTABILIDAD

Capacidad del producto o componente de ser transferido de forma efectiva y eficiente de un entorno hardware, software, operacional o de utilización a otro. Esta característica se subdivide a su vez en las siguientes subcaracterísticas:

- **Adaptabilidad.**
- **Capacidad para ser instalado.**
- **Capacidad para ser reemplazado.**

5.3 MÉTRICA 3

5.3.1 PRÁCTICAS

Métrica 3 reconoce los siguientes tipos de pruebas, en su libro de Técnicas y Prácticas, como prácticas:

- Unitarias
- Integración
- Sistema
- Implantación
- Aceptación
- Regresión

Además, en este mismo libro, reconoce las revisiones formales y revisiones técnicas dentro de prácticas.

En Métrica 3, el Plan de pruebas se comienza a definir en el proceso ASI, Análisis de Sistemas de Información, se especifica detalladamente en el proceso DSI, Diseño de Sistemas de Información y se ejecutarán en el proceso CSI, Construcción de Sistemas de Información.

5.4 ISTQB INTERNATIONAL SOFTWARE TESTING QUALIFICATIONS BOARD

International Software Testing Qualifications Board, en adelante ISTQB. Es un Comité internacional que suministra planes de estudios y glosarios para distintos niveles sobre los que se definen las guías para la acreditación y evaluación de los profesionales de pruebas.

6 TIPOS DE PRUEBA

Un grupo de actividades de pruebas pueden tener por objetivo verificar el sistema de software (o parte del sistema) en base a un motivo u objetivo específico para probar.

Un tipo de prueba se centra en un objetivo de prueba en particular, en función del objeto de prueba se encuentra la siguiente clasificación:

- **Pruebas funcionales:** Una función a realizar por el software,
- **Pruebas no Funcionales:** Una característica de calidad no funcional, tales como la fiabilidad o la usabilidad.
- **Pruebas estructurales:** La estructura o arquitectura del software o sistema.
- **Pruebas de Confirmación o re-testing:** Confirmar que se han solucionado los defectos.
- **Pruebas de Regresión:** Están asociadas a la fase de mantenimiento del sistema y su objetivo es comprobar que los cambios efectuados sobre un componente del sistema de información no introducen un comportamiento no deseado o errores adicionales en los componentes no modificados.

7 NIVELES DE PRUEBA

7.1 PRUEBAS DE COMPONENTES O UNITARIAS

Las pruebas de componente, pruebas unitarias, pruebas de módulo o pruebas de programa, tienen por objeto localizar defectos y comprobar el funcionamiento de módulos de software, programas, objetos, clases, etc., que pueden probarse por separado.

Pueden realizarse de manera aislada del resto del sistema, en función del contexto del ciclo de vida de desarrollo y del sistema. Para ello pueden utilizarse stubs, controladores y simuladores.

7.2 PRUEBAS DE INTEGRACIÓN

Las pruebas de integración se ocupan de probar las interfaces entre los componentes, las interacciones con distintas partes de un mismo sistema, como el sistema operativo, el sistema de archivos y el hardware, y las interfaces entre varios sistemas.

Para las pruebas de integración se establecen dos estrategias:

- Integración incremental
- Integración no incremental o Big-bang.

Las posibles estrategias de integración incremental son:

- a) **Estrategia top-down**, descendente, o de arriba a abajo. Consiste en empezar la integración y la prueba por los módulos que están en los niveles superiores. Se necesitan módulos auxiliares que sustituyan a los módulos que se llaman desde los de alto nivel.

- b) **Estrategia bottom-up**, ascendente, o de abajo a arriba. Consiste en empezar la integración y prueba por los módulos que están en los niveles inferiores. En este caso se necesitan módulos conductores que simulen las llamadas a los que se están probando.

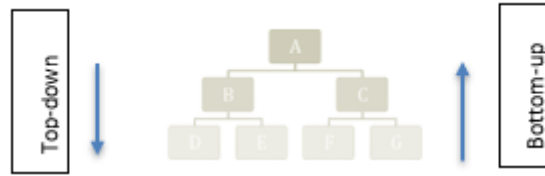


Imagen 7: Top-down y Bottom-up

Pruebas de integración no incremental o Big-bang. Consiste en integrar y probar todo al mismo tiempo. Con esta prueba se detectan muchos errores, pero es complicado identificar las causas que los producen.

7.3 PRUEBAS DE SISTEMA

Las pruebas de sistema se refieren al comportamiento de todo un sistema/producto. En las pruebas de sistema, el entorno de pruebas debe coincidir en la máxima medida posible con el objetivo final o con el entorno de producción a fin de minimizar el riesgo de no identificar fallos específicos del entorno durante las pruebas.

Las pruebas de sistema deben estudiar los requisitos funcionales y no funcionales del sistema y las características de calidad de los datos a probar.

Dentro de este nivel de prueba se encuentran:

- **Pruebas de seguridad.**
- **Pruebas de rendimiento.**
- **Pruebas de recuperación.**
- **Pruebas de resistencia o stress.**

7.4 PRUEBAS DE ACEPTACIÓN

Las pruebas de aceptación son a menudo responsabilidad de los clientes o usuarios de un sistema, a pesar de que también pueden participar otros implicados.

El objetivo principal de las pruebas de aceptación no es localizar defectos, sino que es evaluar la buena disposición de un sistema para su despliegue.

Las pruebas de aceptación pueden adoptar, entre otras, las siguientes formas:

Pruebas de aceptación de usuario

En general, verifican la idoneidad de uso del sistema por parte de los usuarios de negocio.

Pruebas de aceptación operativas

La aceptación del sistema por parte de los administradores del sistema

Pruebas de aceptación contractual y normativa

Las pruebas de aceptación contractual toman como base los criterios de aceptación previstos en un contrato y la normativa aplicable.

Pruebas alfa y beta (o de campo)

Las pruebas alfa se llevan a cabo en el emplazamiento de la organización de desarrollo, pero no las realiza el equipo de desarrollo. Las pruebas beta, o pruebas de campo, las realizan los clientes o clientes potenciales en sus propias instalaciones.

8 TÉCNICAS DE PRUEBAS (ISTQB Y MÉTRICA3)

Existen distintas técnicas para el diseño de pruebas definidas dentro de las medidas analíticas de aseguramiento de la calidad, según ISTQB. Se clasifican en: técnicas estáticas, aquellas que no implican la ejecución del software y técnicas dinámicas, si su aplicación conlleva que el software sea ejecutado.

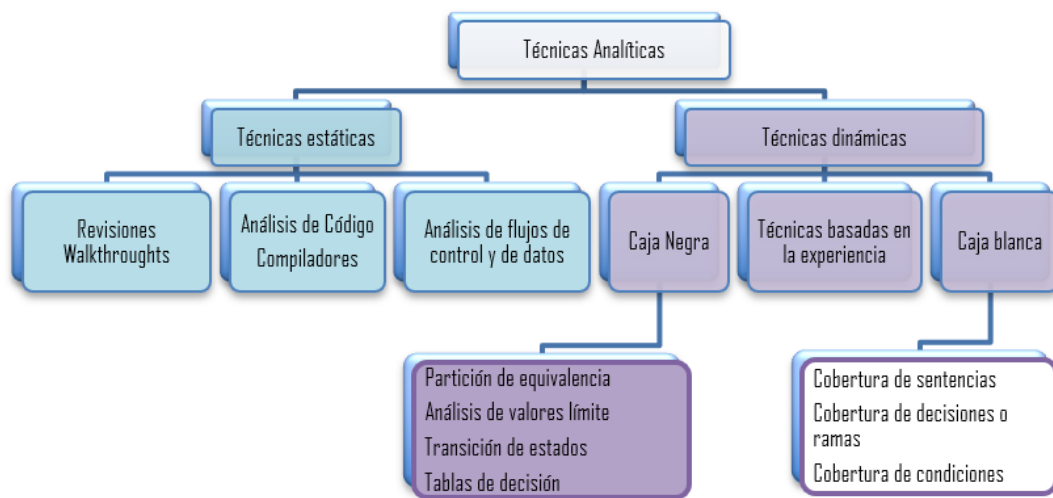


Imagen 8: Técnicas Analíticas o Medidas QA – Clasificación ISTQB

8.1 TÉCNICAS ESTÁTICAS

Revisiones/ Walkthroughs

Auditorías: investigación para determinar el grado de cumplimiento y la adecuación de los procedimientos, estándares u otros requisitos establecidos, y la efectividad de la implementación realizada. Hay 3 tipos: auditoría del producto (según requisito), auditoría del proceso (de desarrollo) del proyecto o su gestión y auditoría del sistema de garantía de calidad.

Revisiones: reunión formal en la que se presenta el estado actual de un proyecto a los interesados. Se realizan al principio del desarrollo (auditoría al final). Pueden ser formales (públicas, se informa de los resultados por escrito) o informales y hay dos tipos: inspecciones (se revisa un checklist de comprobación) y walkthroughs (simulación del funcionamiento con casos de prueba y ejemplos).

Según su alcance, el Plan General de Garantía de Calidad del Consejo Superior de Administración Electrónica, distingue entre:

- Revisiones Mínimas (RM – examen de las características + visibles de los documentos).
- Revisiones Técnicas Formales (RTF – evaluar que los documentos son conformes a las especificaciones).

- Inspecciones Detalladas (ID - detección de cualquier tipo de defectos que puedan afectar a un documento obtenido al final de una fase de desarrollo, la diferencia con RTF es que RTF comprueba adecuación e ID busca defectos).

En Métrica V3, se indica otra clasificación:

- Revisiones Formales (rigurosas: no se aplican siempre porque puede bloquear el proyecto, detectan defectos: que no satisface especificaciones).
- Revisiones Técnicas (comprobar adecuación con especificaciones). Pressman sólo indica
- Revisiones Técnicas Formales (detectar errores, no según especificaciones o estándares, sw uniforme, proyecto manejable).

Verificación formal: demostrar matemáticamente la corrección de un programa según especificaciones. Alto coste sólo para sistemas críticos.

Análisis de código y Compiladores

Con el uso de herramientas para la realización de análisis estático (compiladores, analizadores) el código del programa puede ser objeto de inspección sin ser ejecutado. Permiten evaluar la complejidad estructural, conduciendo a una estimación del esfuerzo en pruebas esperadas y conocer la corrección de la sintaxis del código sin necesidad de ejecución.

Análisis de control de flujo y de datos

Permiten la detección de ramas muertas, código inalcanzable y anomalías en los datos.

8.2 TÉCNICAS DINÁMICAS

Son aquellas que requieren la ejecución del objeto que se está probando o de un modelo del mismo, se clasifican en tres tipos:

Métodos basados en la estructura o de Caja Blanca: la estructura interna del objeto de prueba es utilizada para diseñar los casos de prueba (código/sentencias, menús, llamados, etc.). El porcentaje de cobertura es medido y utilizado como fuente para la creación de casos de prueba adicionales.

- Pruebas de sentencias y cobertura.
- Pruebas de decisiones y cobertura.

Métodos basados en la especificación o de Caja Negra: el objeto de prueba ha sido seleccionado de acuerdo con el modelo funcional de software.

- Partición de equivalencia.
- Análisis de valores límite.
- Prueba de tabla de decisión.
- Pruebas de transición de estado.
- Pruebas de caso de uso.

Métodos basados en la experiencia: el conocimiento y experiencia respecto de los objetos de prueba y su entorno son las fuentes para el diseño de casos de prueba, constituye otra fuente de información.

8.2.1 PRUEBAS DE CAJA BLANCA

- **Cobertura de sentencias**
Diseño de casos de prueba suficientes para que cada sentencia en el programa se ejecute, al menos, una vez.

COBERTURA SENTENCIA= N.º SENTENCIAS EJECUTADAS EN LA PRUEBA / N.º TOTAL SENTENCIAS

- **Cobertura de condición**

Diseño de casos de prueba suficientes para que cada condición en una decisión tenga una vez resultado verdadero y otro falso.

COBERTURA CONDICIÓN= N.º CONDICIONES EJECUTADAS EN LA PRUEBA / N.º TOTAL CONDICIONES

Cobertura Decisión/Condición: se escriben casos de prueba suficientes para que cada condición en una decisión tome todas las posibles salidas, al menos una vez, y cada decisión tome todas las posibles salidas, al menos una vez.

Cobertura de Condición Múltiple: se escriben casos de prueba suficientes para que todas las combinaciones posibles de resultados de cada condición se invoquen al menos una vez.

El 100% de cobertura de sentencia no implica el 100% de cobertura de condición.

- **Cobertura de Caminos:**

Se escriben casos de prueba suficientes para que se ejecuten todos los caminos de un programa. Entendiendo camino como una secuencia de sentencias encadenadas desde la entrada del programa hasta su salida.

COBERTURA CAMINOS = N.º CAMINOS EJECUTADOS EN LA PRUEBA / N.º TOTAL CAMINOS

8.2.2 PRUEBAS DE CAJA NEGRA

Partición de Equivalencia (EP) (Clases de equivalencia)

Permite seleccionar datos de prueba y se pueden aplicar a todos los niveles de pruebas. Los valores de entrada del software o del sistema se dividen en los grupos que se espera demuestren un comportamiento similar. Las particiones de equivalencia (Clases) son aplicables a: valores válidos, inválidos y valores de salida.

Pruebas de Análisis de valores límite (BVA)

Permite seleccionar datos de prueba y se pueden aplicar a todos los niveles de pruebas. Los límites constituyen un área en el que las pruebas tenderán a incluir defectos. Presenta una mayor detección de defecto que el método de Clases de Equivalencia. Los valores máximos y mínimos de una partición son sus valores límites.

Tablas de decisión

A través de tablas, se representan los requisitos del sistema que contienen condiciones lógicas y de documentar el diseño del sistema interno. Se deben incluir combinación de salidas, situaciones o eventos.

Pruebas de transición de estados

Estas técnicas ayudan al diseño de la prueba ya que los casos de pruebas podrán cubrir: la secuencia estados, cualquier estado, cualquier transición y la secuencia de transiciones.

Pruebas de caso de uso

El caso de uso contiene la descripción de cómo debe usarse el sistema, especificado desde la perspectiva de usuarios de negocio. Esta técnica contempla su uso en el diseño de la prueba de modo que, se contemplen los procesos generales, alternativas, pre-condiciones post-condiciones y roles aplicables.

9 HERRAMIENTAS DE PRUEBAS DE SOFTWARE

	OPEN SOURCE	COMERCIALES
GESTIÓN DE PRUEBAS	○ Bugzilla Testopia ○ FitNesse ○ qaManager ○ qaBook ○ RTH (open source) ○ Salome-tmf ○ Squash TM ○ Test Environment Toolkit ○ TestLink ○ Testitool ○ XQual Studio ○ Radi-testdir ○ Data Generator	○ HP Quality Center/ALM ○ QA Complete ○ qaBook ○ T-Plan Professional ○ SMARTS ○ QAS.Test Case Studio ○ PractiTest ○ SpiraTest ○ TestLog ○ ApTest Manager ○ Zephyr
PRUEBAS FUNCIONALES	○ Selenium ○ Cypress ○ Appium (Mobile Web Apps, native or hybrid) ○ Postman ○ Soapui ○ NUnit (○ Watir (Pruebas de aplicaciones web en Ruby) ○ WatiN (Pruebas de aplicaciones web en .Net) ○ Capedit ○ Canoo WebTest ○ Solex ○ Imprimatur ○ SAMIE ○ ITP ○ WET ○ WebInject	○ QuickTest Pro ○ Rational Robot ○ Sahi ○ SoapTest ○ Test Complete ○ QA Wizard ○ Squish ○ vTest ○ Internet Macros
PRUEBAS DE CARGA Y RENDIMIENTO	○ FunkLoad ○ FWPTT load testing ○ loadUI ○ JMeter	○ HP LoadRunner ○ LoadStorm ○ NeoLoad ○ WebLOAD Professional ○ Forecast ○ ANTS – Advanced .NET Testing System ○ Webserver Stress Tool ○ Load Impact
PRUEBAS UNITARIAS	○ XUnit: JUnit, NUnit, RUnit, PHPUnit... ○ TestNG: Creado para suplir algunas deficiencias en JUnit. ○ NUnit (.Net) ○ JTiger: Basado en anotaciones, como TestNG. ○ CppUnit ○ Visual Studio UnitTesting	

Tabla 1 Herramientas

